Lab 6: Code Coverage and Branch Coverage

Name: Sampath
SRN: PES1UG24CS416
Section: G
Course: Software Engineering Laboratory

1. Aim

To write Python test cases for the Self Storage program (self_storage.py) and measure statement (code) coverage and branch coverage using Coverage.py and pytest, aiming for at least 90%.

2. Files Maintained

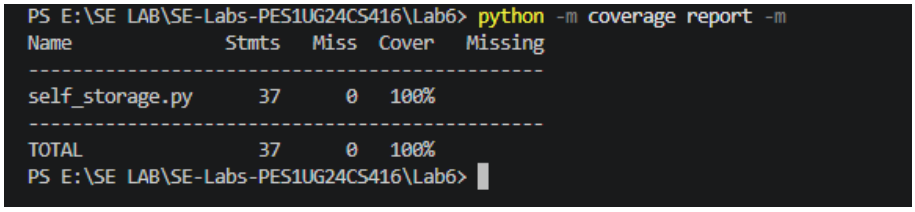
self_storage.py - program under test
CodeCoverageTest_51_self_storage.py - test cases for code coverage (16 tests)
BranchCoverageTest_51_self_storage.py - test cases for branch coverage (24 tests)
GitHub: https://github.com/Sampath-404/SE-Labs-PES1UG24CS416/tree/main/Lab6

3. Commands Used

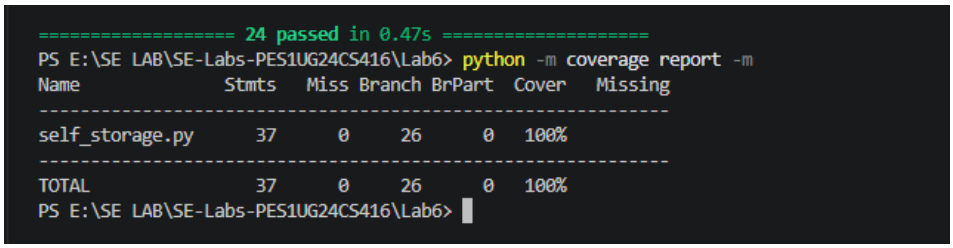
```
Code coverage:
python -m coverage run --source=self_storage -m pytest \
    CodeCoverageTest_51_self_storage.py -v
python -m coverage report -m

Branch coverage:
python -m coverage run --branch --source=self_storage -m pytest \
    BranchCoverageTest_51_self_storage.py -v
python -m coverage report -m
```

4. Final Coverage Results

Code coverage: 37 statements, 0 missed, **100%**. All 16 tests passed.



Branch coverage: 37 statements, 26 branches, 0 missed, 0 partial, **100%**. All 24 tests passed.



5. Program Under Test: self_storage.py

```
DEPOSIT_MONTHS = {"small": 1, "medium": 2, "large": 3}

def rent_due(unit_size, months, is_climate, is_member):
    if unit_size not in DEPOSIT_MONTHS:
        raise ValueError("unknown unit size")
    if months < 0:
        raise ValueError("invalid months")
    covered = DEPOSIT_MONTHS[unit_size]
    if is_member and unit_size != "large":
        covered += 1
    extra = months - covered
    if extra <= 0:
        return 0
    if is_climate:
        rate = 120
    else:
        rate = 80
    if extra > 6:
        rate = rate - 10
        if is_climate:
            rate -= 5
    return extra * rate

def access_allowed(hour, visitors, is_owner):
    if is_owner:
        return visitors <= 4
    if hour < 6 or hour > 22:
        return False
    return visitors <= 2

def payment_status(days_to_due, is_autopay, has_prior_late):
    if is_autopay:
        cutoff = 0
    else:
        cutoff = 5
    if has_prior_late:
        cutoff += 10
    if days_to_due >= cutoff:
        return "current"
    elif days_to_due >= cutoff - 15:
        return "grace"
    return "lien"
```

6. Test File for Code Coverage: CodeCoverageTest_51_self_storage.py

```
import pytest
from self_storage import rent_due, access_allowed, payment_status

# RENT DUE TESTS

def test_small_unit_deposit():
    assert rent_due("small", 1, False, False) == 0

def test_unknown_unit():
    with pytest.raises(ValueError):
        rent_due("extra_large", 2, False, False)

def test_negative_months():
    with pytest.raises(ValueError):
        rent_due("small", -1, False, False)

def test_member_discount():
    assert rent_due("small", 3, False, True) == 80

def test_large_member():
    assert rent_due("large", 5, False, True) == 160

def test_climate_control():
    assert rent_due("small", 3, True, False) == 240

def test_long_term_non_climate():
    assert rent_due("small", 8, False, False) == 490

def test_long_term_climate():
    assert rent_due("small", 8, True, False) == 735

# ACCESS TESTS

def test_owner_access():
    assert access_allowed(12, 4, True) is True

def test_non_owner_access():
    assert access_allowed(12, 2, False) is True

def test_access_denied():
    assert access_allowed(12, 3, False) is False

# PAYMENT STATUS TESTS

def test_payment_current():
    assert payment_status(5, False, False) == "current"

def test_payment_grace():
    assert payment_status(-5, False, False) == "grace"

def test_payment_lien():
    assert payment_status(-11, False, False) == "lien"

def test_autopay_with_prior_late():
    assert payment_status(10, True, True) == "current"
def test_access_outside_allowed_hours():
    assert access_allowed(5, 1, False) is False
```

7. Test File for Branch Coverage: BranchCoverageTest_51_self_storage.py

```
import pytest

from self_storage import rent_due, access_allowed, payment_status

# ========================================
# RENT DUE - BRANCH COVERAGE TESTS
# ========================================

def test_unknown_unit():
    with pytest.raises(ValueError):
        rent_due("invalid", 2, False, False)

def test_negative_months():
    with pytest.raises(ValueError):
        rent_due("small", -1, False, False)

def test_member_true():
    assert rent_due("small", 3, False, True) == 80

def test_member_false():
    assert rent_due("small", 3, False, False) == 160

def test_large_member():
    assert rent_due("large", 5, False, True) == 160

def test_extra_zero():
    assert rent_due("small", 1, False, False) == 0

def test_extra_positive():
    assert rent_due("small", 3, False, False) == 160

def test_climate_true():
    assert rent_due("small", 3, True, False) == 240

def test_climate_false():
    assert rent_due("small", 3, False, False) == 160

def test_extra_greater_than_six():
    assert rent_due("small", 8, False, False) == 490

def test_extra_not_greater_than_six():
    assert rent_due("small", 7, False, False) == 480

def test_long_term_climate():
    assert rent_due("small", 8, True, False) == 735

# ========================================
# ACCESS - BRANCH COVERAGE TESTS
# ========================================

def test_owner_access_allowed():
    assert access_allowed(23, 2, True) is True

def test_owner_access_denied():
    assert access_allowed(12, 5, True) is False

def test_non_owner_access():
    assert access_allowed(12, 2, False) is True

def test_before_opening():
    assert access_allowed(5, 1, False) is False

def test_after_closing():
    assert access_allowed(23, 1, False) is False

def test_visitor_limit_exceeded():
    assert access_allowed(12, 3, False) is False

# ========================================
# PAYMENT STATUS - BRANCH COVERAGE TESTS
# ========================================

def test_autopay_true():
    assert payment_status(0, True, False) == "current"
def test_autopay_false():
```

```
    assert payment_status(5, False, False) == "current"

def test_prior_late_true():
    assert payment_status(10, True, True) == "current"

def test_current_status():
    assert payment_status(5, False, False) == "current"

def test_grace_status():
    assert payment_status(-5, False, False) == "grace"

def test_lien_status():
    assert payment_status(-11, False, False) == "lien"
```

8. Conclusion

Both test suites cover every statement and every branch of self_storage.py, giving 100% code coverage and 100% branch coverage, which is above the required 90%.